

Phase 4 写真機能の本格化 仕様書

現場 OS Lite / 要件定義 + 実装仕様書（最小工数）

作成日：2026 年 6 月 23 日 / 目的：建設会社が現場写真を実用レベルで管理できる

概要

既存の写真機能（現場別アップロード・グリッド表示）を、**複数選択・全画面拡大・撮影日時・現場別ギャラリー**まで引き上げる。**DB 変更なし・新規依存なし**で実装でき、Supabase の手作業も不要。

スコープ

実装する	実装しない
複数写真（ライブラリ複数選択）	AI 解析
写真拡大表示（スワイプ・ピンチズーム）	GPS / 位置情報
撮影日時表示（= created_at）	動画
写真一覧改善（枚数・新しい順）	PDF
現場別ギャラリー	写真削除・オフライン同期（別フェーズ）

※「撮影日時」は created_at（=アップロード日時）で表示。真の EXIF 撮影日時は新カラム+解析が必要なため本フェーズ非対象。
※ ROADMAP 元 Phase 4 のオフライン同期/geo は再スコープで除外。

1. ユーザーストーリー

- ライブラリから**複数枚をまとめて**現場に追加したい。
- 写真を**タップで拡大し、左右スワイプ**で次々確認したい。
- 各写真が**いつのものか（日時）**を知りたい。
- 現場ごとの**ギャラリー**で整理して見返したい（枚数も把握）。
- 引き続き**自社の写真のみ**（RLS 維持）。

2. DB 変更有無

変更なし（マイグレーション不要・Supabase 手作業なし）。複数写真は既存テーブルに複数行で対応済み、拡大/日時/整理は 既存カラム（path / created_at）と署名付き URL で実現。新規カラム・テーブル・依存なし。

3. 画面一覧

画面	区分	主な要素
現場詳細の写真セクション	変更	枚数表示／タイル tap で拡大／追加（カメラ=1 枚・ライブラリ=複数）／「すべて見る」→ギャラリー
写真ギャラリー（現場別）	追加	現場の全写真をグリッド表示（新しい順・枚数・空/エラー・pull-to-refresh）
写真拡大ビューア	追加	全画面。PageView で左右スワイプ+InteractiveViewer でピンチズーム。日時と「n / N」表示

4. Repository 構成

既存 listPhotos / uploadPhoto(1枚) / createSignedUrl を流用。**複数アップロードは Controller で uploadPhoto をループ**（Repository は増やさない）。createSignedUrls(一括署名) は任意・性能用で MVP 不要。ImagePickerService に pickMultiple()（image_picker の pickMultiImage）を追加 = **新規依存なし**。

5. Riverpod 構成

新規 Provider 追加なし（既存構成の再利用を最優先）。ギャラリー/ビューアとも既存 2 つのみで実装可能：

- ・ photosProvider.family(siteId) :一覧・PageView 元データ・枚数 (photos.length) ・日時 (createdAt) ・refresh 対象
- ・ photoUrlProvider.family(path) : 各タイル/各ページの署名付き URL

PhotoUploadController に addPhotos() をメソッド追加 (複数選択→ループ upload→invalidate、結果に追加/失敗枚数)。既存 photoUploadControllerProvider 流用、既存 addPhoto (カメラ 1 枚) は維持。ビューアの現在 index は画面ローカル (PageController)。photoCountProvider は作らない。

6. 完了条件

#	条件
1	複数写真を一度に追加できる
2	写真をタップで全画面表示・スワイプ切替・ピンチズーム
3	各写真に日時 (created_at) が表示される
4	現場別ギャラリーで新しい順+枚数表示
5	RLS 維持 (自社のみ表示・アップロード)
6	flutter analyze / flutter test 緑
7	dev 実機確認成功

7. テスト条件

- ・ ImagePickerService(Fake) : pickMultiple() が複数/空を返す
- ・ PhotoUploadController.addPhotos : 複数成功 (upload 回数=件数) / 空 (未アップロード) / 全失敗 (state エラー)
- ・ PhotoRepository(Fake) : uploadPhoto 呼び出し回数・一覧反映
- ・ 既存 43 件が緑のまま。拡大/ギャラリーの画像描画は実機確認。

■ 実装仕様（最小工数）

追加・変更ファイル

区分	ファイル
変更	image_picker_service.dart (pickMultiple) / photo_upload_controller.dart (addPhotos) / site_detail_screen.dart (写真セクション) / app_routes.dart・app_router.dart (ルート) / test/photos/fakes.dart
追加	photo_gallery_screen.dart (P-Gallery) / photo_viewer_screen.dart (P-Viewer) / test/photos/photo_upload_controller_multi_test.dart
なし	DB マイグレーション / 新規依存 / Supabase 手作業

ImagePickerService (追加メソッド)

```
Future<List<Uint8List>> pickMultiple() async {
  final files = await _picker.pickMultiImage(maxWidth: 1600, imageQuality: 80);
  final out = <Uint8List>[];
  for (final f in files) { out.add(await f.readAsBytes()); }
  return out; // キャンセル時は空
}
```

PhotoUploadController (追加メソッド)

```
typedef PhotosAddResult = ({int uploaded, int failed, bool cancelled});

Future<PhotosAddResult> addPhotos({required String siteId, required String companyId}) async {
  state = const AsyncValue.loading();
  final list = await ref.read(imagePickerServiceProvider).pickMultiple();
  if (list.isEmpty) { state = const AsyncValue.data(null); return (uploaded:0, failed:0, cancelled:true); }
  var ok = 0, ng = 0; Object? err; StackTrace? st;
  for (final bytes in list) {
    try { await ref.read(photoRepositoryProvider)
      .uploadPhoto(siteId: siteId, companyId: companyId, bytes: bytes); ok++; }
    catch (e, s) { ng++; err = e; st = s; }
  }
  ref.invalidate(photosProvider(siteId));
  state = (ng > 0 && ok == 0) ? AsyncValue.error(err!, st!) : const AsyncValue.data(null);
  return (uploaded: ok, failed: ng, cancelled: false);
}
```

画面

- ・ **現場詳細セクション**：ヘッダ 写真 (N) + 「すべて見る」 + 「追加」。追加ボトムシート＝カメラ(1 枚 addPhoto)/ライブラリ(複数 addPhotos)。タイル tap→ビューア。
- ・ **P-Gallery** PhotoGalleryScreen(siteId): photosProvider(siteId) を .when。RefreshIndicator+GridView (3 列・新しい順)。空/エラー対応。タイル tap→ビューア。
- ・ **P-Viewer** PhotoViewerScreen(siteId, initialIndex): 黒背景。PageView.builder (initialPage=initialIndex, onPageChanged で index 更新) +各ページ InteractiveViewer+photoUrlProvider(path)→Image.network。AppBar= 「n / N」、下部に yyyy/MM/dd HH:mm (created_at.toLocal、null は —)。

ルート

```
static const sitePhotos = '/sites/:id/photos';
static const sitePhotoView = '/sites/:id/photos/:index';
// '/photos' を '/photos/:index' より先に登録。遷移は context.push('/sites/$siteId/photos/[$index]')
```

実装手順

1. pickMultiple 追加 → Fake 更新 (analyze 緑) → addPhotos 追加
2. P-Gallery / P-Viewer 追加 → ルート登録
3. 現場詳細セクション改修 (枚数・tap 拡大・複数追加・すべて見る)
4. テスト (addPhotos) → analyze / test 緑
5. iOS ビルド → 実機で 複数追加 / 拡大スワイプ / 日時 / ギャラリー 確認 → commit/push

工数見積もり

区分	目安
🔧 実装（コード一式＋テスト＋analyze/test 緑＋push）	約 35～50 分
👤 実機確認（複数追加・拡大・ギャラリー）	約 5 分
DB / Supabase 手作業	なし
合計（壁時計）	約 1 時間以内

関連: docs/Phase4 写真本格化要件定義.md / docs/Phase4 写真本格化実装仕様書.md / 踏襲元 = Phase 1.3 写真管理。